

CIS241

System-Level Programming and Utilities

C Stack and Heap

Erik Fredericks, frederer@gvsu.edu

Fall 2025

Based on material provided by Erin Carrier, Austin Ferguson, and Katherine Bowers

Stack and heap

Two areas of memory

- The stack is a stack data structure
- The heap is not necessarily a heap 🙄
- Variables are in one or the other
 - Which one depends on how variable is declared!
- So far, we've only dealt with the stack

Stack

New block (scope) for each function called

- Function local variables are stored in block
- When function returns, that block is freed
 - Those variables are gone!

Heap

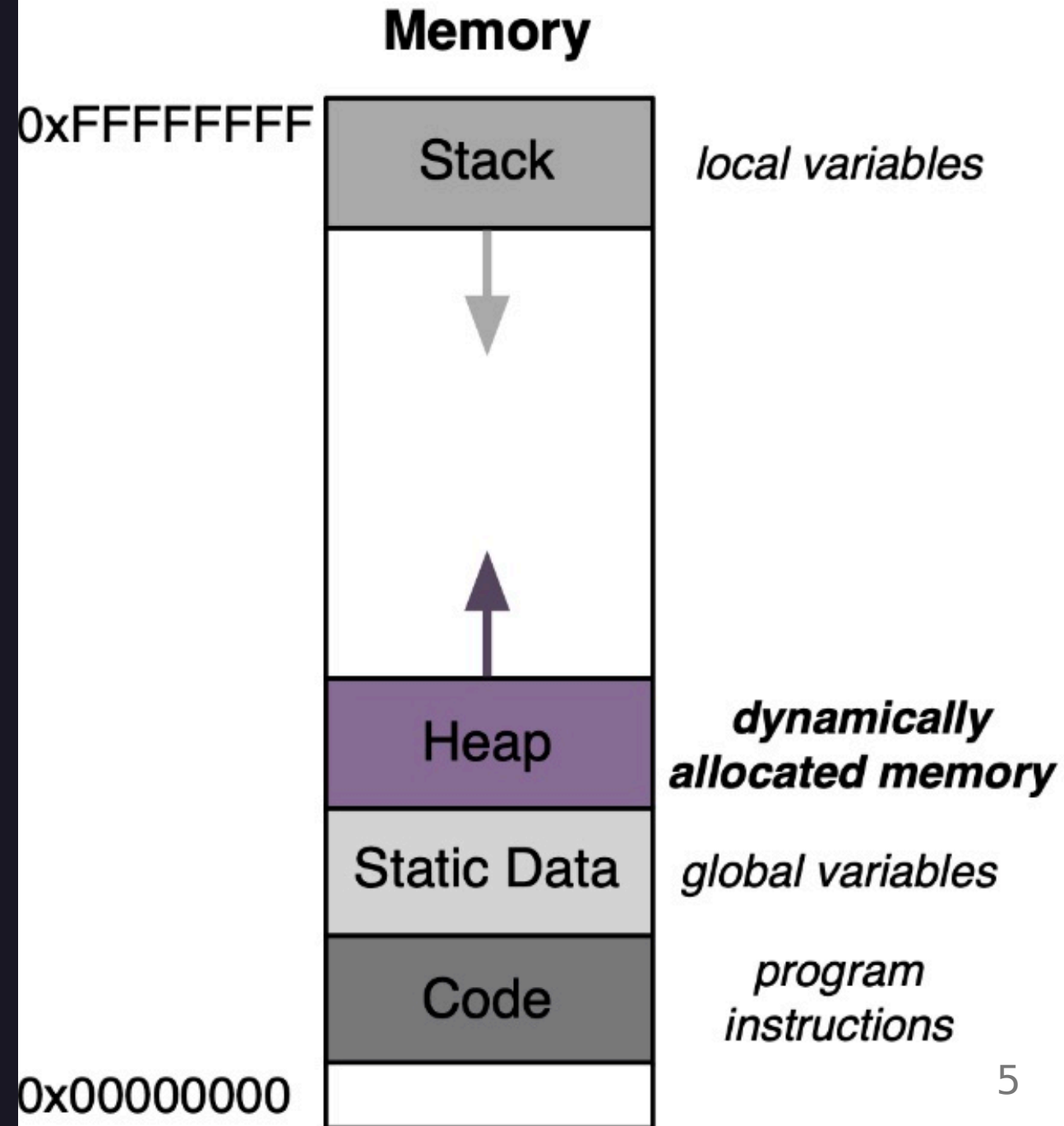
Dynamically allocated memory

- Not tied to a particular functions scope
- Can be allocated and freed at any time

We must free any memory we allocate

- What happens if not?
 - Memory leaks!
 - You can lose access (pointer) to allocated memory
 - Thus your program can't free it
 - Usually cleaned up by OS when program exits

Stack vs. Heap



So, why?

Stack allocates / deallocates automatically

- Often limited in size
- All allocations on stack are static (sizes are set at compile time)

Heap allows flexibility, but require more thought

- Larger arrays
- Arrays that persist after a function returns
- Amount of memory is unknown at compile time
 - *Technically possible in stack via VLAs
 - But we're ignoring this :)

Memory allocation!

Stack

- Static (compile time) memory allocation
- Examples: `int x;` `char arr[20];`

Heap

- Dynamic (runtime) memory allocation
- Examples: `malloc();` `realloc();` etc.
 - This is our next lecture :)

